Fixed-Path Isolation for Structural Tool-Output Compression

Abdallah Alwarawreh
Manuscript v0.3 — 3 August 2026
abood@tokz.dev

Abstract

Long structural tool results make language-model agents expensive, but a single selector can also delete the policy and task text that governs them. We describe a deterministic, extractive boundary for JSON, JSONL, CSV, and log tool output. It labels each segment as protected, control, or data. Protected policy and caller-authorized task input are fixed byte passthrough; only structural data is eligible for selection under the remaining soft aggregate byte target. This is a small compositional API invariant, not a new general non-interference theorem: changing data cannot change the fixed-path result because that branch never receives data.

On 500 synthetic, query-aligned tool-output cases, role isolation retained all fixed text and the two exact target-evidence strings at a 22.49% mean aggregate kept-byte ratio. A ratio-matched role-erasure ablation retained the evidence but not the fixed policy. These results show a narrow tradeoff—role isolation can preserve fixed context without reducing exact evidence on this benchmark—not general answer quality, token savings, or prompt-injection resistance. Prose, chat, RAG, and unknown data pass through unchanged, so the aggregate target can be exceeded substantially.

1 Introduction

Tool-using language-model agents often append large structured tool results to instructions and caller-authorized task input. Long inputs increase inference cost and latency, motivating prompt compression [11–13, 16]. Tool output is also an instruction-bearing data channel: indirect prompt injection places instructions in data that an application later presents to a model [8, 23, 24]. Existing benchmarks show that the security and utility of agents must be evaluated separately [3].

Most prompt-compression work asks which tokens are useful to a downstream task. That question is incomplete when inputs cross trust boundaries. If policy, task intent, and untrusted data enter one selector under one budget, then data can influence whether higher-priority text survives. Model-level instruction hierarchies and structured-query defenses address whether a model follows lower priority instructions [2, 21]; they do not specify what a compressor may delete before the model sees the prompt. Liu et al. identify this shared-budget attack surface and propose isolated compression for trusted and untrusted regions [14].

This paper studies a narrower systems property. The reference implementation separates prompt regions before compression. It treats protected policy and control text as fixed, then compresses only structural tool output into verifiable source-byte spans.

The contributions are:

- an explicit role and assembly contract: protected policy and fixed authorized task input are byte-exact source spans, while structural tool data is separately eligible for selection;
- a deterministic, extractive reference implementation that reports rather than hides soft-target overflow; and
- a controlled synthetic structural-tool-output study and robustness tests, reported as an ablation of role isolation rather than a comparison with state-of-the-art compressors.

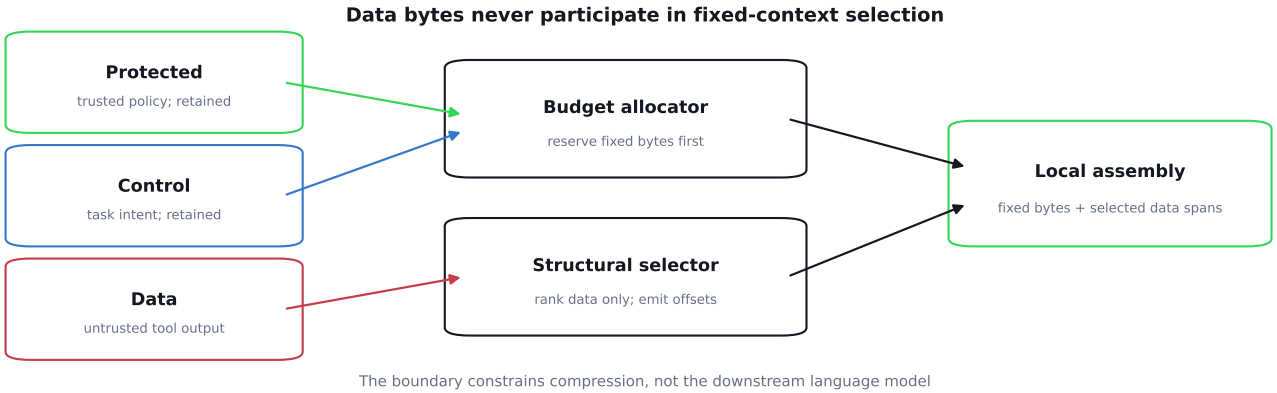

Figure 1: COMPRESS-NI separates role assignment from data selection. Protected and control bytes bypass the selector. Structural data is selected independently and the client assembles the result from source-byte spans.

2 Threat Model and Fixed-Path Invariant

2.1 Inputs and adversary

Let a request be $R = (P, C, D, q, b)$. P is an ordered set of protected segments, C an ordered set of control segments, D an ordered set of data segments, q caller-authorized task intent, and b an optional aggregate byte budget. Each segment carries an identifier, a role, a trust label, UTF-8 text, and an optional compression policy. Protected segments must be labeled trusted. *Control* means caller-authorized task input that is fixed for selection; it is not a claim that its source is trusted or privileged.

The adversary may choose or mutate the text of data segments, including their length, syntax, and instruction-like content. The adversary cannot change roles, trust labels, P, C, q, b, the compressor implementation, or the source hashes verified by the client. Correct segmentation and an authorized query are therefore trusted-computing-base assumptions. This follows the least-privilege and separation principles of classical protection systems [18], but the three roles are operational categories, not a general information-flow lattice [5].

The downstream language model and its actions are outside the property. In particular, the adversary may still induce model behavior if malicious text survives data compression. Stronger agent architectures constrain control and capability flows across model calls and tools [4, 6]; COMPRESS-NI does not claim those guarantees.

2.2 Fixed-path invariant

For every completed request with the same ordered protected and control segments, the reference implementation emits the same full UTF-8 byte interval, source hash, and materialized bytes for each fixed segment, regardless of data text. This is a direct consequence of the API composition, not a deep security result. Its practical value is that callers and SDKs can test and audit the boundary rather than relying on a relevance selector to preserve authority.

3 Design

Figure 1 shows the request path. The implementation accepts between one and 128 uniquely identified segments. It rejects a protected segment that is not trusted and rejects invalid budgets before selection. The hosted API schema validates role and trust enums; TypeScript clients receive the same constraint statically. An untyped local JavaScript caller that bypasses schema validation is part of the trusted computing base.

3.1 Offset-only fixed path

For every protected or control segment s, the engine constructs a full span map $[(0, |s|)]$, where offsets are half-open UTF-8 byte indices, stores the SHA-256 digest of the source bytes, and marks the segment as passthrough. No selected text is generated on this path. The public clients check response order, identifier, role, trust label, byte

length, digest, role totals, and exact materialized fixed text before returning a result. Thus the source, rather than server-returned text, remains authoritative for assembly.

3.2 Budget allocation and data selection

Let B_P, B_C, and B_U denote bytes in protected, control, and non-structural data, respectively. If an aggregate budget b exists, the available structural-data budget is

$$B_D = \max(0, b - B_P - B_C - B_U). \tag{1}$$

The implementation distributes B_D among structural data segments proportionally to their source-byte sizes. Each such segment is then routed to the existing deterministic structural compressor with q and its segment policy. Data bytes never become a query and never enter the fixed path.

Structural outputs are sorted, non-overlapping source spans. The client concatenates only those source bytes. The current firewall version deliberately passes prose, chat, RAG, unknown, and otherwise unsupported data through as full offset maps. This avoids a semantic path that returns selected text and joining spaces, but it also means prose cannot be reduced and malicious prose is not removed.

Budgets are soft because syntactic scaffolding and required spans can exceed a target. The engine never drops fixed context to force compliance. Instead it reports either a fixed-context overflow or a data-constraint overflow. The implementation's current external reason string calls the first case `protected_content_exceeds_budget`, although control bytes are also included; this naming mismatch is an API limitation, not a different policy.

3.3 Why the property holds

For each $s \in P \cup C$, the implementation branches on s's role before any compression call and invokes a retain function whose text-valued input is only s. That function deterministically routes s for metadata, hashes s's UTF-8 bytes, and emits the full interval $[0, |s|)$. Neither D, the data-available budget, nor any data selection result is an argument. Changing D to D' can change data routes, per-data allocations, aggregate kept bytes, and the aggregate reason, but not any fixed segment result. Since P, C, and their order are fixed, their projected results and materialized bytes are equal.

This is intentionally simple; the empirical question is whether parsing, byte handling, validation, or integration breaks that composition in practice. It is not a proof of the TypeScript runtime, dependencies, SDK, or provider wrappers. It also does not cover data–data allocation interference or availability.

4 Evaluation

We ask whether role isolation preserves fixed context without reducing query-aligned exact evidence on a synthetic structural benchmark, what its soft-target and local-latency characteristics are, and whether deterministic robustness tests expose fixed-path failures.

4.1 Protocol

The main benchmark contains five synthetic tool-result domains (jobs, deployments, support tickets, builds, and shipments), four row counts (32, 80, 200, and 500), and 25 deterministic seeds per cell, for 500 cases. Each case has one known failed row with a unique marker and a query that names its failure condition. This is a query-aligned structural-selection task, not a representative agent-workload sample. We pair a clean payload with instruction-like and malformed data mutations to exercise implementation paths; templates and positions vary by seed and are not a factorial attack study. The firewall's nominal soft aggregate target is fixed across each clean–mutated pair: all protected and control bytes plus 20% of the clean data bytes. The query states the caller's task and failure condition but does not contain the hidden marker used to score evidence retention.

We measure exact protected/control byte retention, equality of the serialized fixed projections across each pair, and exact target evidence survival (both the unique marker and failure message). Evidence retention is a deterministic selection metric, not downstream question-answering accuracy.

We compare with two intentionally simple policies. *Flat compression* places protected, control, and data objects into one JSON array and invokes the same structural engine. For each case, a deterministic target search matches its achieved aggregate ratio to the firewall result; we report the residual ratio error. Fixed objects rotate among

Table 1: Mean retention over 500 paired cases. "Target" requires two exact task-evidence strings. Ratios are proportions; higher is better.

Method	Protected	Control	Target	Kept ratio
COMPRESS-NI	1.000	1.000	1.000	0.2249
Flat compression	0.000	0.826	1.000	0.2272
Reserve+head	1.000	1.000	0.200	0.2249

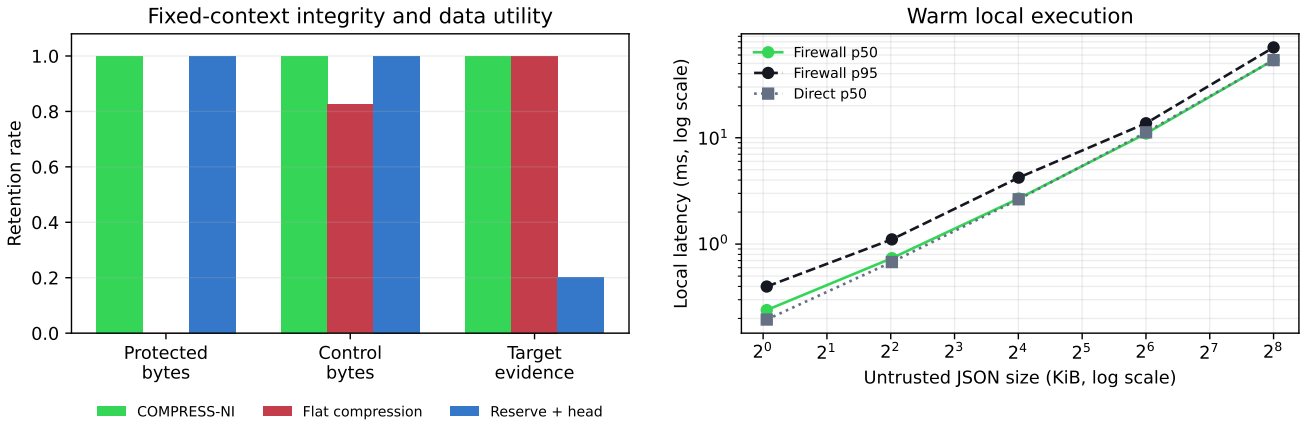

Figure 2: Left: role and evidence retention at matched achieved ratios. Right: warm local latency for COMPRESS-NI and direct data-only structural compression. Values are from one host and one process.

head, middle, and tail positions. It tests the effect of removing only the role boundary, not a realistic competitor. *Reserve+head* retains all fixed text, then fills a data prefix to the firewall's achieved byte count. It satisfies fixed preservation but does not perform relevance selection. These are mechanism ablations, not claims against external prompt-injection defenses.

A second seeded robustness test runs 2,000 data-only perturbations over explicitly routed JSON, JSONL, CSV, log, prose, and unknown inputs with multilingual fixed text. JSON, JSONL, CSV, and log fixtures contain 60 valid records so their structural selectors execute; prose and unknown inputs exercise documented passthrough paths. We report method counts by format. We replay 250 identical inputs and hash the complete serialized result. Latency uses 20 warm-up calls followed by 200 alternating measurements per size, with direct compression receiving the same 20% data-byte target as the firewall's data segment. The benchmark ran on Windows 10.0.26200, Node.js 20.17.0, an AMD Ryzen 5 7535HS (12 logical CPUs), and 39.2 GiB of visible memory. All compression ran locally on the JavaScript main thread; no language model or network call was used.

4.2 Integrity and utility

Table 1 and Figure 2 show the principal results. COMPRESS-NI's fixed projection was unchanged in all 500 paired cases and both fixed roles had full byte retention. These deterministic grid results do not estimate real-world attack prevalence.

The role-erasure ablation retained the queried evidence in every case, but did so while dropping all protected bytes and 17.4% of control bytes on average. This is a fixture- and selector-specific result: protected retention was zero at head, middle, and tail positions, while control retention was 0.839, 0.819, and 0.819, respectively. Its mean absolute ratio mismatch was 0.34 percentage points and the maximum was 0.97 points. This result does not establish superior selection: both methods retain the engineered target. It shows only that relevance alone does not encode authority. Reserve+head protects authority but retains target evidence only when it happens to fall in the first fifth of the data. On this deliberately query-aligned synthetic task, role separation plus relevance selection retains both. We do not infer general task quality from this benchmark.

The cross-format stress test found zero violations in 2,000 deterministic trials: 363 JSON, 334 JSONL, 343 CSV, 319 log, 322 prose, and 319 unknown. Every JSON, JSONL, CSV, and log trial used structural selection; every prose and unknown trial used passthrough. All 250 full-result replays were byte-identical on the tested host

Table 2: Warm local latency in milliseconds (200 runs per size after 20 warmups).

Data KiB	Firewall p50	Firewall p95	Direct p50	Direct p95
1.04	0.240	0.399	0.195	0.294
4.05	0.737	1.107	0.678	1.010
16.06	2.683	4.218	2.639	3.989
64.05	10.963	13.694	11.251	13.967
256.01	53.831	70.715	53.726	72.555

and runtime. Cross-platform identity remains untested; neural and numerical software can vary across platforms even for fixed inputs [19], although the firewall path measured here is structural and does not invoke the semantic classifier.

4.3 Soft-budget behavior

COMPRESS-NI kept 22.49% of aggregate source bytes on average, compared with 22.72% for the ratio-matched flat baseline. Structural scaffolding caused the result to exceed the aggressive soft target in 469 of 500 cases (93.8%). The mean overshoot was 37.6 bytes and the 95th percentile was 71 bytes. This is expected policy behavior—fixed and structurally required bytes outrank a soft target—but it matters operationally. A caller requiring a hard provider limit must reserve headroom or handle the reported constraint rather than assuming exact budget compliance. Unsupported prose is operationally worse: it is retained whole, so the output is approximately 1×, 5×, or 20× a 256-byte soft target when prose is supplied at those sizes. The implementation reports `data_constraints_exceed_budget`; it does not silently truncate prose. Byte retention is serialized model-visible text size, not provider token count or cost, which depends on the provider tokenizer and surrounding message framing.

4.4 Latency

Table 2 reports warm in-process latency. At 64 KiB, median firewall latency was 10.96 ms; at 256 KiB it was 53.83 ms. The direct baseline compresses only the data segment, while the firewall additionally routes, hashes, and accounts for two fixed segments. Alternating call order reduces systematic warm-cache bias, but differences of this size should not be treated as stable overhead estimates from a single process.

Input sizing, routing, parsing, and hashing require passes over the bytes. Candidate ranking includes a sort, and the current ordered insertion strategy can incur quadratic movement in the number of candidates in the worst case. We therefore make no universal linear-time claim from the five-point latency curve.

5 Related Work

Prompt and context compression. Selective Context prunes low-information lexical units [13]; LLMLingua and LongLLMLingua use learned, budgeted, query-aware token selection [11, 12]; and LLMLingua-2 casts compression as extractive token classification [16]. RECOMP trains extractive and abstractive compressors for retrieval-augmented language models [22]. These systems primarily optimize task quality and cost. COMPRESS-NI instead asks which source domains are eligible for selection at all. Its structural output is less expressive than learned compression but is byte-extractive and locally verifiable. Semi-extractive QA likewise uses copied spans to make factual support easier to inspect [20].

Prompt-injection defenses. Spotlighting transforms untrusted data to signal provenance [9]. StruQ separates prompt and data channels and trains the model to follow only the prompt channel [2]. Instruction Hierarchy trains models to prioritize privileged instructions [21]. Spotlighting and StruQ both include pre-inference transformations; their objective is downstream instruction/data separation. COMPRESS-NI instead specifies which trust domains a compressor may select from, and cannot substitute for those downstream defenses.

CaMeL separates a privileged planner from a quarantined model, propagates capabilities, and mediates tool calls [4]. The LLMbda calculus gives agent values provenance labels and proves a substantially stronger probabilistic non-interference result [6]. System design patterns similarly emphasize explicit trust boundaries and constrained

execution [1]. COMPRESS-NI contributes one small, composable boundary inside such an architecture: untrusted data cannot alter which fixed context survives compression.

The closest predecessor is Liu et al.'s adversarial information-loss analysis and Isolated Compression defense, which compresses trusted and untrusted regions separately before recombination [14]. COMPRESS-NI does not claim the isolation policy as novel. Its increment is to make trusted regions fixed byte passthrough, emit client-verifiable source offsets, state the result as FSNI, and expose overflow when fixed content exceeds a soft budget. Twin Agent addresses a broader architecture: an untrusted exploration agent produces a bounded, model-generated context residual for a privileged safe agent [10]. COMPRESS-NI is a narrower deterministic primitive that can sit inside a privilege-separated agent design.

Information flow and provenance. FSNI borrows the counterfactual shape of non-interference from Goguen and Meseguer [7] and later language-based information-flow work [17]. Unlike confidentiality policies, our observable is explicitly an integrity-sensitive selection result and data length remains visible. Source hashes and byte intervals form lightweight derivation evidence; we use provenance in its ordinary sense and do not claim conformance with the W3C PROV data model [15].

6 Limitations and Security Considerations

Not downstream injection prevention. Data selected for relevance can contain attacker instructions, and prose data passes through intact. The reported COMPRESS-NI benchmark contains no language-model call and reports no attack-success rate. The repository also includes a separate opt-in downstream agent harness with synthetic tool transcripts; its provider-dependent results are preliminary and are not evidence of complete injection prevention.

Trusted labeling and intent. A caller can mislabel data as protected or supply a query derived from adversarial content. The wire protocol does not authenticate semantic origin. Provider integrations reduce integration effort but remain part of the trusted computing base.

Concrete integration failures include concatenating tool output into a control segment, labeling arbitrary user text as protected, deriving the query from tool text, promoting nested tool fields into control, or flattening roles before the call. The implementation cannot repair these errors: fixed retention and trusted origin are distinct properties. Provider wrappers may also fail open by sending the original request after a compression failure; callers that need fail-closed behavior must enforce it outside this component.

Data–data interference. Structural data segments share the remaining budget. A large segment or a non-structural segment that must pass through can reduce another data segment's allocation. FSNI covers only protected and control outputs.

Availability behavior. Fixed context can exceed the requested budget, and the caller must handle the explicit constraint. Some provider wrappers fall back to the original uncompressed request on compression or validation failure. That preserves availability and fixed text but does not enforce a fail-closed downstream security policy. A data-path exception may also reject the whole request; termination and availability are outside FSNI.

Hosted confidentiality. In hosted mode, Tokz receives all request text, including protected policy, to compute span maps. The service is payload-stateless and retains no source text; structural responses contain offsets rather than source text. Usage and billing records do persist, and offset-only output does not mean the payload never crossed the network. Local or private execution is required when that transfer is unacceptable.

Evaluation scope. Workloads are synthetic, query-aligned, and use simple exact-evidence probes. The flat and reserve+head baselines are ablations, not state-of-the-art security systems. We measured one operating system, runtime, CPU, and process. External corpora, adaptive attacks, cross-platform determinism, multi-segment allocation, and downstream task outcomes remain future work.

7 Artifacts, Disclosure, and Conclusion

The public companion repository, github.com/tokz-dev/compress-ni, includes a compact result summary, figures, this manuscript, and a public hosted benchmark driver that uses the published Tokz SDK. The driver deterministically generates the synthetic fixtures and reruns the functional comparison and fixed-path stress checks against the current hosted service:

```
python -m pip install -r requirements.txt
TOKZ_API_KEY=... python reproduce.py --full --output results/hosted-full.json
```

```
python analyze.py --input results/hosted-full.json
```

The hosted rerun sends these synthetic payloads to Tokz and consumes the caller's credits. Pinning the SDK does not pin the service because the API does not expose an engine revision, so a later run may differ from the frozen paper result. It also cannot reproduce the in-process latency measurements: an HTTP timing includes network, authentication, metering, and shared-service latency. The compression engine remains commercial Tokz product code and is not distributed. The author develops Tokz and has a financial interest in the system. The frozen COMPRESS-NI results use generated operational records, no human-subject data, and no paid language-model call.

Structural tool-output compression is not only a relevance problem when inputs cross trust boundaries. The reference implementation makes one narrow policy explicit: fixed policy and authorized task text are not candidates for selection, regardless of data. This is an auditable composition rule, not a complete prompt-injection firewall or a claim about prose compression.

References

[1] Luca Beurer-Kellner, Beat Buesser, Ana-Maria Creţu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naeff, Ezinwanne Ozoani, Andrew Paverd, Florian Tramèr, and Václav Volhejn. Design patterns for securing LLM agents against prompt injections. *arXiv preprint arXiv:2506.08837*, 2025. doi: 10.48550/arXiv.2506.08837.

[2] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium*, pages 2383–2400, 2025. URL `https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe`.

[3] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[4] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025. doi: 10.48550/arXiv.2503.18813.

[5] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976. doi: 10.1145/360051.360056.

[6] Zac Garby, Andrew D. Gordon, and David Sands. The LLMbda calculus: AI agents, conversations, and information flow. *arXiv preprint arXiv:2602.20064*, 2026. doi: 10.48550/arXiv.2602.20064.

[7] Joseph A. Goguen and José Meseguer. Security policies and security models. In *1982 IEEE Symposium on Security and Privacy*, pages 11–20, 1982. doi: 10.1109/SP.1982.10014.

[8] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pages 79–90, 2023. doi: 10.1145/3605764.3623985.

[9] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. Defending against indirect prompt injection attacks with spotlighting. In *Proceedings of the Conference on Applied Machine Learning in Information Security*, volume 3920 of *CEUR Workshop Proceedings*, pages 48–62, 2024. URL `https://ceur-ws.org/Vol-3920/paper03.pdf`.

[10] Zhanhao Hu, Dennis Jacob, Xiao Huang, Zhaorun Chen, Bo Li, and David Wagner. Twin agent: Context residual compression for privilege separated agents. *arXiv preprint arXiv:2607.19595*, 2026. doi: 10.48550/arXiv.2607.19595.

[11] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, 2023. doi: 10.18653/v1/2023.emnlp-main.825.

[12] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 1658–1677, 2024. doi: 10.18653/v1/2024.acl-long.91.

[13] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353, 2023. doi: 10.18653/v1/2023.emnlp-main.391.

[14] Zesen Liu, Zhixiang Zhang, Yuchong Xie, and Dongdong She. When compression becomes an attack surface: Black-box attacks on prompt-compressed LLM agents. *arXiv preprint arXiv:2510.22963v4*, 2025. doi: 10.48550/arXiv.2510.22963. Revised 2026.

[15] Luc Moreau and Paolo Missier. PROV-DM: The PROV data model. W3c recommendation, World Wide Web Consortium, 2013. URL `https://www.w3.org/TR/2013/REC-prov-dm-20130430/`.

[16] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 963–981, 2024. doi: 10.18653/v1/2024.findings-acl.57.

[17] Andrei Sabelfeld and Andrew C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003. doi: 10.1109/JSAC.2002.806121.

[18] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.

[19] Alex Schlögl, Nora Hofer, and Rainer Böhme. Causes and effects of unanticipated numerical deviations in neural network inference frameworks. *Advances in Neural Information Processing Systems*, 36, 2023. doi: 10.52202/075280-2446.

[20] Tal Schuster, Adam D. Lelkes, Haitian Sun, Jai Gupta, Jonathan Berant, William W. Cohen, and Donald Metzler. SEMQA: Semi-extractive multi-source question answering. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 1363–1381, 2024. doi: 10.18653/v1/2024.naacl-long.74.

[21] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024. doi: 10.48550/arXiv.2404.13208.

[22] Fangyuan Xu, Weijia Shi, and Eunsol Choi. RECOMP: Improving retrieval-augmented LMs with context compression and selective augmentation. In *International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=mlJLVigNHp`.

[23] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and defending against indirect prompt injection attacks on large language models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 1809–1820, 2025. doi: 10.1145/3690624.3709179.

[24] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, 2024. doi: 10.18653/v1/2024.findings-acl.624.